

ACTUADORES Y SENSORES

RETO 1 – MAPEO DE TRAYECTORIA CON ROBOT XRP

ESTUDIANTES:
DAVID ZULUAGA HENAO
LUCAS ARANGO BOTERO
NICOLÁS ZAPATA JURADO

PROFESOR:
FELIPE MENDOZA GIRALDO

FECHA:
2 DE OCTUBRE DE 2026

UNIVERSIDAD EAFIT
MEDELLÍN
2026

TABLA DE CONTENIDO

1. INTRODUCCIÓN.....	2
1.1. OBJETIVOS.....	2
1.2. DESCRIPCIÓN DE LA PISTA.....	2
2. DESCRIPCIÓN DEL SISTEMA.....	3
2.1. SENSORES Y ACTUADORES.....	3
2.2. ARQUITECTURA DEL PROGRAMA.....	4
3. SEGUIMIENTO DE LÍNEA Y LÍNEA TRANSVERSAL.....	4
3.1. SEGUIDOR DE LÍNEA.....	4
3.2. CONTROL DE VELOCIDAD POR RUEDA.....	5
3.3. DETECCIÓN DE LA LÍNEA TRANSVERSAL.....	5
4. MODELO DE PREDICCIÓN (ENCODERS + IMU).....	5
4.1. RUIDO DE PROCESO Q.....	6
5. ESTIMACIÓN DE MOVIMIENTO CON EL LiDAR.....	6
5.1. CONVERSIÓN DEL BARRIDO A PUNTOS.....	7
5.2. PL-ICP (PUNTO A LÍNEA).....	7
6. FILTRO DE KALMAN EXTENDIDO.....	7
6.1. VECTOR DE ESTADO Y PREDICCIÓN.....	7
6.2. CORRECCIÓN CON EL LiDAR.....	8
6.3. MATRICES Q Y R ELEGIDAS Y SU CALIBRACIÓN.....	8
7. RESULTADOS.....	9
7.1. DATOS REGISTRADOS POR EL ROBOT.....	9
7.2. UBICACIÓN DE LOS DATOS SOBRE EL PLANO DE LA PISTA.....	10
7.3. DATOS REUBICADOS SOBRE LA PISTA.....	12
7.4. ANÁLISIS DE RESULTADOS.....	14
8. CONCLUSIONES.....	14
8.1. MEJORAS PROPUESTAS.....	15
9. REFERENCIAS.....	15
10. ANEXO: ARCHIVOS ENTREGADOS.....	15

1. INTRODUCCIÓN

El reto consiste en programar el robot diferencial XRP para que siga una línea oscura sobre una pista clara, estime en tiempo real su posición (x, y) y su orientación θ fusionando tres fuentes de información —encoders de los motores, IMU (giroscopio) y un LiDAR de barrido 360°— mediante un filtro de Kalman, y guarde el recorrido completo en un archivo de texto. Una línea transversal define el origen del sistema de coordenadas ($x = 0$, $y = 0$, $\theta = 0$) y también la meta: el robot debe detenerse al cruzarla de nuevo después de completar la vuelta.

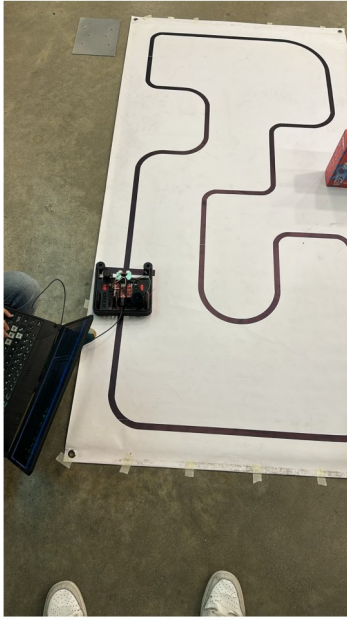
En este informe se describe la solución implementada en el programa `reto1_xrp.py` (versión v14), el modelo de predicción con encoders e IMU, la estimación de movimiento con el LiDAR, el filtro de Kalman extendido (EKF) con sus matrices Q y R , y los resultados obtenidos en la pista, incluyendo la ubicación de los datos medidos sobre el plano real del recorrido.

1.1. OBJETIVOS

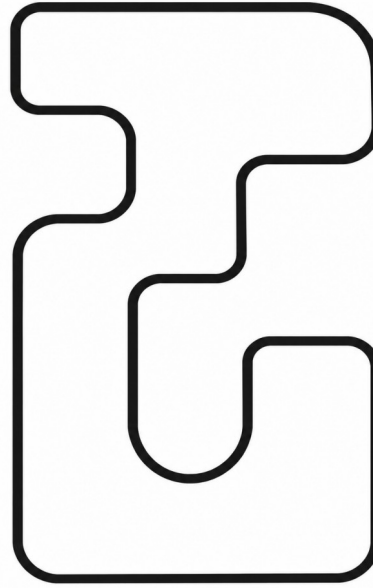
- Seguir la línea de la pista de forma estable con control de velocidad independiente en cada rueda.
- Detectar la línea transversal de inicio y de fin con los dos sensores de reflectancia.
- Predecir la pose del robot con encoders e IMU (modelo cinemático del robot diferencial).
- Estimar el desplazamiento y el giro del robot comparando barridos del LiDAR (scan matching).
- Fusionar ambas fuentes con un filtro de Kalman extendido y generar el archivo `datos.txt`.
- Analizar la trayectoria obtenida y ubicarla sobre el plano de la pista.

1.2. DESCRIPCIÓN DE LA PISTA

La pista es una lona blanca con una línea negra cerrada que combina rectas largas, curvas de 90° y una curva en U (Figura 1). El recorrido de la línea mide **99,4 cm de ancho × 198,5 cm de largo**; con estas medidas, el perímetro de la línea central es de unos **811 cm** (sección 7.2). La línea transversal de salida/meta está sobre la recta larga del lado izquierdo, y el robot recorre la pista en sentido horario visto desde arriba.



a)



b)

Figura 1. Pista del reto: a) fotografía de la pista con el robot XRP sobre la línea; b) plano de la pista en vista superior.

2. DESCRIPCIÓN DEL SISTEMA

2.1. SENSORES Y ACTUADORES

La Tabla 1 resume cada componente, cómo quedó configurado y para qué se usa dentro de la solución.

Tabla 1. Componentes del robot XRP y su papel en el reto.

Componente	Configuración usada	Uso en la solución
Motores con encoder (x2)	12 CPR $\times$ 48,75 de reducción = 585 cuentas por vuelta de rueda; radio de rueda $r = 3,0$ cm; separación entre ruedas $L = 15,5$ cm.	Actuador de tracción y sensor de odometría (distancia recorrida Δs). Control PI de velocidad por rueda a 100 Hz.
Sensores de reflectancia (x2)	Blanco: 0,562 (izq.) / 0,508 (der.). Negro: 0,873 / 0,862. Línea transversal: ambos $\geq 0,80$.	Seguimiento de línea (error lateral) y detección de la línea transversal de inicio y fin.
IMU (giroscopio eje z)	Calibración de 2 s con el robot quieto + estimación del sesgo residual (100 muestras).	Giro $\Delta\theta$ en la predicción del EKF y amortiguamiento del giro en el seguidor.
LiDAR 360° (LDROBOT, tipo LD06)	UART a 230 400 baudios, paquetes de 47 bytes (12 puntos, CRC-8). Lecturas válidas entre 12 cm y 600 cm.	Estimación del desplazamiento y del giro entre barridos (PL-ICP) $\rightarrow$ corrección del EKF.
Sensor de distancia (ultrasonido)	Disponible en la plataforma.	No se usó en la estimación final (la pista no tiene obstáculos).

A diferencia de lo planteado en la guía, el LiDAR se conectó **directamente** a la placa XRP (puerto Qwiic 0: IO4 = TX, IO5 = RX) y no a través del ESP32. Por eso el programa incluye su propio lector del protocolo LDROBOT: busca la cabecera 0x54 0x2C, valida el CRC-8 (polinomio 0x4D) de cada paquete, descarta puntos de baja intensidad y arma una vuelta

completa de 360 distancias en milímetros. Al arrancar, el programa prueba los puertos disponibles y, si no encuentra el LiDAR, continúa solo con encoders e IMU.

2.2. ARQUITECTURA DEL PROGRAMA

El programa corre en MicroPython sobre la placa SparkFun XRP (RP2350) con la librería XRPLib. Para que la marcha nunca se vea afectada por los cálculos de estimación, se separó en tres niveles de tiempo:

- **Control de la marcha (100 Hz):** lectura de encoders y sensores de línea, seguidor de línea y PI de velocidad de cada rueda. Tiene su propio reloj y se atiende entre todos los demás cálculos.
- **Estimación (10 Hz, $\Delta t = 0,1$ s):** paso de predicción del EKF con encoders + giroscopio y registro de Δs , $\Delta \theta$ y Δt de cada ciclo.
- **LiDAR en modo diferido:** durante el recorrido solo se guardan “fotos” del LiDAR (una vuelta completa cada 5 cm recorridos, cada 10° girados o cada 1,5 s, hasta 160 barridos de 720 bytes). Al llegar a la meta el robot se detiene, ejecuta de nuevo el EKF ciclo a ciclo con los datos guardados aplicando cada barrido en su instante real (scan matching + corrección) y escribe los archivos.

Esta decisión se tomó porque el scan matching tarda decenas de milisegundos en MicroPython y, al ejecutarlo en vivo, el control de las ruedas se retrasaba y el robot se salía de la línea. Por la misma razón, los puntos se guardan en RAM y se escriben en la memoria flash solo al final: escribir en cada ciclo desactiva las interrupciones varios milisegundos y se pierden muestras del giroscopio y bytes del LiDAR. El resultado del EKF es el mismo que en vivo, pero sin perder ninguna medición. Se generan tres archivos: datos.txt y datos_NicolasZapata.csv (x, y en cm) y un archivo completo con la pose, la covarianza y el estado del LiDAR en cada ciclo.

3. SEGUIMIENTO DE LÍNEA Y LÍNEA TRANSVERSAL

3.1. SEGUIDOR DE LÍNEA

Cada sensor de reflectancia se normaliza con sus propios valores de blanco y negro (medidos con la prueba de calibración p2), y el error lateral se calcula con la diferencia normalizada de los dos sensores:

$$s_i = \min \left(1, \max \left(0, \frac{v_i - v_{i,blanco}}{v_{i,negro} - v_{i,blanco}} \right) \right), e = \frac{s_L - s_R}{\max(s_L + s_R, 1)}$$

El error se filtra (constante de tiempo 0,15 s) y se aplica una zona muerta de $\pm 0,035$. Con él se calculan la velocidad de giro y la de avance deseadas. El término $-K_{yaw} \cdot \omega_{med}$ usa el giroscopio para amortiguar las oscilaciones, y la velocidad baja de 6,0 cm/s en recta a 3,5 cm/s en curva según la magnitud del error:

$$\omega_{ref} = K_p e_f + s a t_{0,08} \left(K_D \dot{e}_f \right) - K_{yaw} \omega_{med}, v_{ref} = v_{recta} - (v_{recta} - v_{curva}) |e_f|$$

$$v_L = v_{ref} - \frac{\omega_{ref} L}{2}, v_R = v_{ref} + \frac{\omega_{ref} L}{2}$$

Con $K_P = 0,55 \text{ rad/s}$, $K_D = 0,010$, $K_{\text{yaw}} = 0,18$ y $|\omega_{\text{ref}}| \leq 0,35 \text{ rad/s}$, además de límites de aceleración y de cambio de giro para que el movimiento sea suave. Si se pierde la línea, el robot recuerda el último lado donde la vio y gira hacia ese lado con una rueda detenida y la otra a 8 cm/s hasta recuperarla.

3.2. CONTROL DE VELOCIDAD POR RUEDA

Los dos motores no responden igual al mismo esfuerzo, así que cada rueda tiene su propio lazo de velocidad medido con su encoder (prealimentación por fricción y velocidad + PI con anti-windup y rampa de esfuerzo):

$$u = k_s + k_v |\dot{v}| + K_p (\dot{v} - v) + K_i \int (\dot{v} - v) dt$$

Tabla 2. Parámetros del control de velocidad de cada rueda.

Parámetro	Rueda izquierda	Rueda derecha
k_s (fricción estática)	0,32	0,40
k_v (esfuerzo por cm/s)	0,022	0,030
K_P / K_I	0,025 / 0,14	0,025 / 0,14
Esfuerzo máximo / rampa	0,85 / $3,0 \text{ s}^{-1}$	0,85 / $3,0 \text{ s}^{-1}$

El control también detecta ruedas bloqueadas, encoders sin pulsos o con el signo invertido, y en ese caso detiene el robot de forma segura y guarda los datos tomados hasta ese momento.

3.3. DETECCIÓN DE LA LÍNEA TRANSVERSAL

La línea transversal es más angosta que la separación entre los sensores, así que solo cuando el robot la cruza **ambos** sensores quedan sobre negro al mismo tiempo. Se considera cruce cuando los dos valores crudos superan 0,80. Para evitar falsos positivos:

- **Salida:** el robot se coloca con los dos sensores sobre la transversal; tras 40 ms de confirmación se define el origen $(x, y, \theta) = (0, 0, 0)$, se reinician el yaw y los encoders y comienza la grabación.
- **Meta:** solo se habilita después de salir de la línea de inicio durante 0,25 s, de recorrer al menos 20 cm y de llevar al menos 3 s en marcha. Luego se exige un cruce de 60 ms para detener el robot.

4. MODELO DE PREDICCIÓN (ENCODERS + IMU)

La distancia que recorre cada rueda en un ciclo se obtiene a partir de los encoders (ecuación (1) de la guía):

$$d_{L,R} = \frac{\Delta \text{ticks}_{L,R} 2\pi r}{N}, r = 3.0 \text{ cm}, N = 585 \text{ cuentas/rev}$$

El avance del centro del robot se toma del promedio de las dos ruedas. Para el giro se usa el **giroscopio**: XRPLib integra el yaw internamente a unos 200 Hz, lo que es más preciso que derivar ω de la diferencia de los encoders (las ruedas patinan en las curvas) o que muestrear la velocidad angular una sola vez por ciclo. El giro de los encoders se guarda solo como referencia:

$$\Delta s_k = \frac{d_R + d_L}{2}, \Delta \theta_k = \psi_k - \psi_{k-1} (\text{giroscopio}), \Delta \theta_k^{enc} = \frac{d_R - d_L}{L}$$

El estado se predice con el modelo cinemático del robot diferencial. En lugar de la integración de Euler de la guía se usa el ángulo en la mitad del intervalo (θ), que es más exacto en las curvas sin costo adicional:

$$\hat{x}_k^- = \begin{pmatrix} x_{k-1} + \Delta s_k \cos \theta \\ y_{k-1} + \Delta s_k \sin \theta \\ \theta_{k-1} + \Delta \theta_k \end{pmatrix}, \dot{\theta} = \theta_{k-1} + \frac{\Delta \theta_k}{2}$$

Como el modelo es no lineal, la covarianza se propaga con su Jacobiano (filtro de Kalman extendido):

$$F_k = \begin{pmatrix} 1 & 0 & -\Delta s_k \sin \theta \\ 0 & 1 & \Delta s_k \cos \theta \\ 0 & 0 & 1 \end{pmatrix}, P_k^- = F_k P_{k-1} F_k^T + Q_k$$

4.1. RUIDO DE PROCESO Q

En vez de una matriz Q constante, el ruido se modela en las entradas del modelo (Δs , $\Delta \theta$) y se lleva al estado con su Jacobiano G. Así Q es prácticamente cero con el robot quieto y crece con lo recorrido y lo girado, que es como se comportan realmente los errores de los encoders y del giroscopio:

$$Q_k = G \begin{pmatrix} \sigma_s^2 & 0 \\ 0 & \sigma_\theta^2 \end{pmatrix} G^T + Q_{min}, G = \begin{pmatrix} \cos \theta & -1/2 \Delta s_k \sin \theta \\ \sin \theta & 1/2 \Delta s_k \cos \theta \\ 0 & 1 \end{pmatrix}$$

$$\sigma_s^2 = K_S |\Delta s_k|, \sigma_\theta^2 = K_G \Delta t + (K_{esc} \Delta \theta_k)^2$$

K_S representa el error del encoder por centímetro recorrido (deslizamiento, cuantización, radio de rueda); K_G la deriva del giroscopio por unidad de tiempo (sesgo residual), y K_{esc} el error relativo de escala del giro. Los valores usados están en la Tabla 3 (sección 6).

5. ESTIMACIÓN DE MOVIMIENTO CON EL LiDAR

La guía propone tres niveles para estimar el movimiento entre dos barridos: (1) solo rotación por corrimiento circular, (2) traslación comparando las distancias al frente y atrás, y (3) scan matching completo. Se implementaron los niveles 1 + 2 (método simple) y el nivel 3 (método icp, el usado en el recorrido), que se eligió porque en la pista el frente y la parte de atrás del robot no siempre ven una pared, mientras que el ICP usa todo el barrido.

5.1. CONVERSIÓN DEL BARRIDO A PUNTOS

Cada distancia válida d_i (entre 120 mm y 6000 mm) en el ángulo α_i se convierte a un punto en el marco del robot, $p_i = (d_i \cos \alpha_i, d_i \sin \alpha_i)$, corrigiendo el desfase angular y el sentido de giro del LiDAR (calibrados con la prueba p4). Para el barrido de referencia se calcula además la normal de la superficie en cada punto a partir de sus vecinos; los puntos en bordes o discontinuidades se descartan.

5.2. PL-ICP (PUNTO A LÍNEA)

Se implementó una versión simplificada del PL-ICP de Censi (2008). Se busca la transformación (t_x, t_y, ϕ) que lleva los puntos del barrido actual sobre las superficies del barrido de referencia, minimizando la distancia de cada punto a la recta tangente de su vecino:

$$\min_{t_x, t_y, \phi} \sum_i \left[n_{j(i)}^T \left(R(\phi) p_i + t - q_{j(i)} \right) \right]^2$$

- **Asociación proyectiva:** el punto transformado se convierte a ángulo y se busca su vecino solo en los índices $j-1, j, j+1$ del barrido de referencia (costo $O(n)$, viable en MicroPython).
- **Gauss-Newton:** 5 iteraciones como máximo, con umbral de rechazo de pares que baja de 25 cm a 5 cm, un punto de cada 4° (≈ 90 puntos) y amortiguamiento de Levenberg.
- **Semilla:** el movimiento que predice la odometría, de modo que el ICP solo refina la estimación.
- **Validación:** se acepta el resultado si hay al menos 30 pares y el 35 % de puntos emparejados, el error RMS es ≤ 4 cm y no difiere de la odometría en más de 15 cm o 20° .
- **Barridos de referencia (keyframes):** se compara contra un barrido de referencia que solo se cambia cuando el robot se ha movido 12 cm o girado 12° , lo que reduce la deriva frente a comparar siempre barridos consecutivos.

La incertidumbre del resultado se estima con la inversa de la hessiana del problema de mínimos cuadrados. Como esa estimación suele ser optimista, se multiplica por 4 y se suma un piso mínimo $R_{\min}$:

$$\Sigma_{ICP} \approx \hat{\sigma}_r^2 (J^T J)^{-1}, R_k = R_{\min} + 4 \Sigma_{ICP}$$

6. FILTRO DE KALMAN EXTENDIDO

6.1. VECTOR DE ESTADO Y PREDICCIÓN

El estado es $\hat{x} = (x, y, \theta)^T$ en cm y radianes, con covarianza P de 3×3 . En cada ciclo de 0,1 s se aplica el paso de predicción de la sección 4 con Q_k variable.

6.2. CORRECCIÓN CON EL LiDAR

El ICP entrega el movimiento del robot en el marco del barrido de referencia. Por eso la medición no se obtiene sumando $(\Delta x, \Delta y, \Delta \theta)$ a la pose, como en la plantilla, sino **componiéndolo** ($\oplus$) con la pose que tenía el robot en el barrido de referencia. Además se añade el pequeño desplazamiento δ medido por la odometría desde el instante del barrido hasta el ciclo actual. Como z está en las mismas unidades del estado, H es la identidad:

$$z_k = x_{ref} \oplus \Delta x_{LiDAR} \oplus \delta, H = I_{3 \times 3}$$

$$y_k = z_k - \hat{x}_k^-, S_k = P_k^- + R_k, K_k = P_k^- S_k^{-1}, \hat{x}_k = \hat{x}_k^- + K_k y_k$$

$$P_k = (I - K_k) P_k^- (I - K_k)^T + K_k R_k K_k^T$$

La covarianza se actualiza con la forma de Joseph, que mantiene P simétrica y definida positiva aun con errores de redondeo en flotantes de 32 bits. Antes de corregir se valida la medición con la distancia de Mahalanobis (χ^2 con 3 grados de libertad al 99 %). En lugar de rechazar toda medición “algo rara”, se le resta peso inflando R ; solo se descarta si es completamente inconsistente:

$$d^2 = y_k^T S_k^{-1} y_k : d^2 \leq 11.34 \Rightarrow R_k ; 11.34 < d^2 \leq 567 \Rightarrow R_k \frac{d^2}{11.34} ; d^2 > 567 \Rightarrow \text{se descarta}$$

6.3. MATRICES Q Y R ELEGIDAS Y SU CALIBRACIÓN

Tabla 3. Matrices del filtro de Kalman y parámetros de ruido.

Matriz / parámetro	Valor	Significado
P_0	diag(0,01 cm ² ; 0,01 cm ² ; 1×10^{-4} rad ²)	En la línea transversal se define el origen: incertidumbre inicial baja.
K_S	0,02 cm ² /cm	Varianza de avance por cm recorrido ($\sigma \approx 0,11$ cm por ciclo a 6 cm/s).
K_G	3×10^{-3} rad ² /s	Deriva del giro ($\sigma \approx 1^\circ$ por ciclo de 0,1 s).
K_esc	0,02	Error relativo de escala del giroscopio (2 %).
Q_min	diag(10^{-4} cm ² ; 10^{-4} cm ² ; 10^{-7} rad ²)	Piso numérico de Q.
R_min	diag(0,05 cm ² ; 0,05 cm ² ; 3×10^{-5} rad ²)	Piso de R del LiDAR ($\sigma \approx 0,22$ cm y $0,3^\circ$).
R_k (ICP)	$R_min + 4 \cdot \Sigma_ICP$, rotada al marco global	Cambia en cada barrido según la calidad del emparejamiento.
R (método simple)	diag(1 cm ² ; 1 cm ² ; 0,05 rad ²)	Menos confiable: el rumbo lo manda el giroscopio.
Validación χ^2	11,34 (atenúa) / 567 (descarta)	3 GDL al 99 %; descarte = 50 × umbral.

Para calibrar, se partió de los valores de la plantilla ($Q = \text{diag}(0,01; 0,01; 0,02)$, $R = \text{diag}(0,05; 0,05; 0,05)$) y se ajustaron con pruebas cortas incluidas en el programa:

- **R (prueba p5):** con el robot quieto se toman barridos consecutivos y se aplica el ICP entre ellos. Como el movimiento real es cero, la dispersión de los resultados da directamente la varianza de la medición; con ella se fijó el piso $R_{\min}$.
- **K_S , r y L (prueba p6):** recorridos rectos de longitud conocida y giros completos sobre el propio eje para ajustar el radio de rueda, la separación entre ruedas y el error por centímetro recorrido.
- **K_G y K_{esc} :** deriva del yaw con el robot quieto después de la calibración y error del giro acumulado en vueltas completas.

Con estos valores, en una recta el filtro confía más en la odometría en posición (Q pequeño) y en el LiDAR para corregir el rumbo, que es la fuente de error que más crece. Si Q es grande frente a R , la corrección del LiDAR domina; si R es grande, manda la predicción. La ganancia K_k hace este balance en cada paso según la incertidumbre acumulada en P .

7. RESULTADOS

7.1. DATOS REGISTRADOS POR EL ROBOT

El robot completó la vuelta siguiendo la línea y se detuvo en la línea transversal. El archivo de salida contiene **1090 puntos (x, y)**. La Figura 2 muestra la trayectoria estimada por el EKF tal como la entregó el robot, con los ejes a la misma escala.

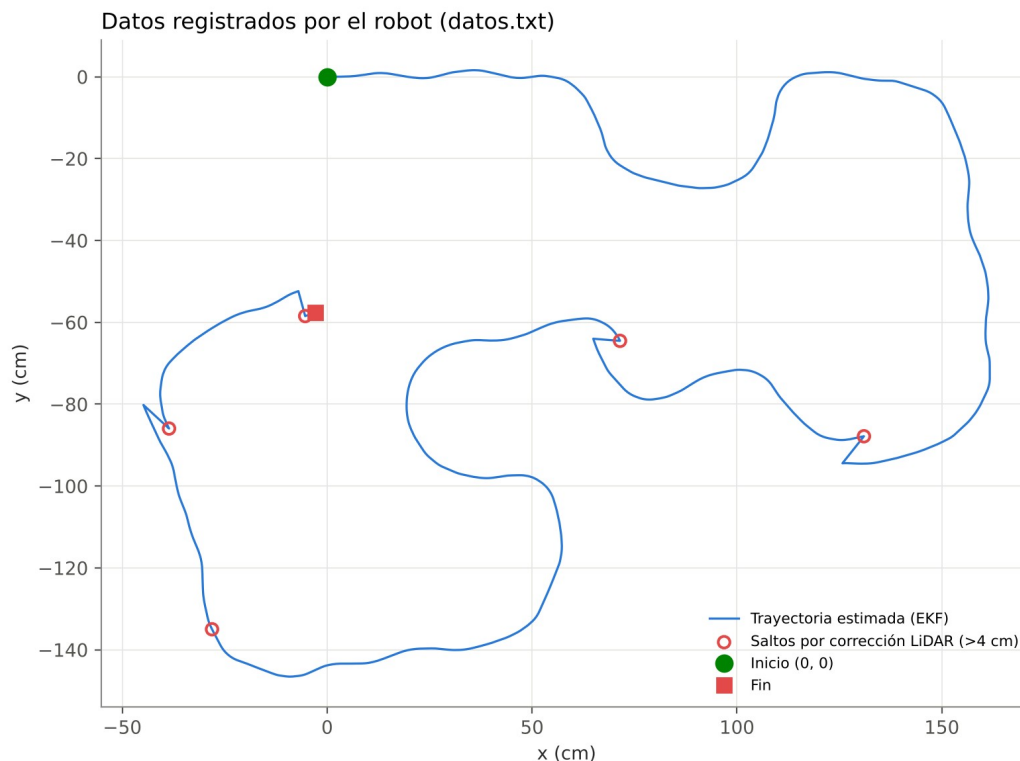


Figura 2. Trayectoria estimada por el EKF (datos del robot). Origen en la línea transversal y eje x en la dirección inicial de avance.

Tabla 4. Resumen de la trayectoria registrada.

Indicador	Valor
Puntos registrados	1090

Indicador	Valor
Distancia recorrida (suma de los pasos)	752 cm
Giro total acumulado	$\approx -360^\circ$ (una vuelta completa en sentido horario)
Extensión de la trayectoria estimada	206,5 cm $\times$ 148,2 cm
Dimensiones reales de la pista (línea)	198,5 cm $\times$ 99,4 cm; perímetro $\approx$ 811 cm
Posición final estimada	(-2,9; -57,6) cm
Error de cierre (distancia final al origen)	57,7 cm (7,7 % de la distancia recorrida)
Salto por corrección del LiDAR mayores a 4 cm	5 (de 4,3 cm a 8,5 cm)

El giro total de -360° muestra que el giroscopio midió correctamente la vuelta completa: no hay una deriva de rumbo grande acumulada al final. Sin embargo, la trayectoria no cierra (la meta es la misma línea de la salida, pero el punto final queda a 57,7 cm del origen) y su tamaño no coincide con la pista. Los saltos marcados con círculos rojos son los momentos en los que la corrección del LiDAR movió la estimación varios centímetros de golpe.

7.2. UBICACIÓN DE LOS DATOS SOBRE EL PLANO DE LA PISTA

En la gráfica obtenida inicialmente los puntos no coincidían con la forma de la pista. Para comparar correctamente los datos con el plano hay que ponerlos en el mismo sistema de coordenadas, con la misma escala y el mismo punto de partida. El procedimiento fue:

- **Línea central de la pista:** se extrajo del plano en vista superior (Figura 1b) como el punto medio entre el borde interior y el exterior de la línea negra.
- **Escala:** el plano en vista superior **no está dibujado a escala:** su relación largo/ancho es 1,58 (1142 px / 723 px), mientras que la pista real mide 198,5 cm $\times$ 99,4 cm (relación 2,0). Por eso, al dibujar los puntos sobre la imagen con una sola escala nunca podían coincidir. Se usó una escala distinta en cada eje: 0,1375 cm/px a lo ancho y 0,1738 cm/px a lo largo. Con ella, el perímetro de la línea central es de 811 cm.
- **Punto de salida y sentido:** se comparó la “firma” de giros de los datos (rumbo en función de la distancia recorrida) con la de la pista para todas las posiciones de salida posibles. La mejor coincidencia ubica la salida en la recta larga izquierda, a unos 54 cm de la esquina inferior, con el robot avanzando hacia arriba (sentido horario), lo que concuerda con la fotografía.
- **Sistema de coordenadas:** la pista se giró para que su dirección de salida coincidiera con el eje +x del robot y su punto de salida con el origen (0, 0), igual que en datos.txt.

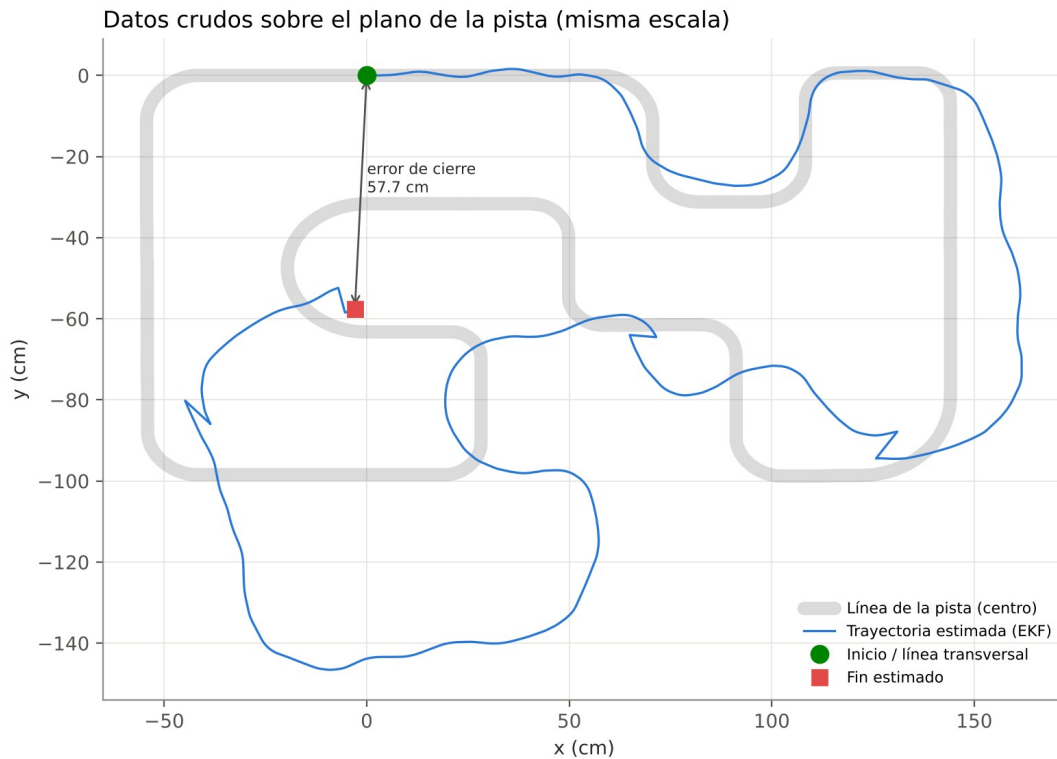


Figura 3. Datos crudos del EKF sobre el plano de la pista a la misma escala. La flecha indica el error de cierre.

La Figura 3 muestra que los primeros 170 cm del recorrido (salida, primera curva y escalón superior) coinciden con la pista con un error medio de 4,5 cm (máximo 6,8 cm). A partir de la curva grande de la derecha la trayectoria estimada se va abriendo: el robot “cree” que está más lejos del origen de lo que realmente está. La Figura 4 cuantifica este efecto como la distancia entre cada punto estimado y el punto de la pista que le corresponde según la distancia recorrida.



Figura 4. Error de posición de la estimación respecto a la pista a lo largo del recorrido.

Tabla 5. Error de posición a lo largo del recorrido.

Distancia recorrida	100 cm	200 cm	300 cm	400 cm	500 cm	600 cm	700 cm
Error de posición	5,0 cm	12,7 cm	39,8 cm	34,2 cm	44,4 cm	54,5 cm	64,8 cm

El error medio en toda la vuelta fue de 31,3 cm (RMS 37,4 cm) y el máximo de 65,4 cm, cerca del final. El error crece sobre todo entre los 170 y los 300 cm, en la curva grande de la derecha y la entrada a la zona central, y vuelve a subir alrededor de los 400 cm. La corrección del LiDAR en 305 cm reduce el error de casi 40 cm a 22 cm, pero la de 395 cm lo aumenta: no todas las correcciones del ICP fueron buenas. Además, la odometría registró 752 cm de recorrido (704 cm sin el zigzag del seguidor), frente a los 811 cm de la línea central. Parte de la diferencia se debe a que el robot recorta las curvas, pero también indica que los encoders subestiman la distancia.

7.3. DATOS REUBICADOS SOBRE LA PISTA

Como el robot fue un seguidor de línea y nunca se salió de la pista, se sabe que su posición real siempre estuvo sobre la línea. Aprovechando eso, se aplicó un post-procesamiento de **emparejamiento con el mapa** (*map matching*):

- Se calcula el rumbo de los datos y el de la línea central en función de la distancia recorrida.
- Con alineamiento temporal dinámico (DTW) se busca la correspondencia más coherente entre ambos perfiles de rumbo, con salida y llegada en la línea transversal. Esto compensa que el robot estime algunos tramos más largos o más cortos de lo real.
- Cada punto medido se lleva a su posición correspondiente sobre la línea central, conservando la pequeña oscilación lateral del seguidor ($\pm 2,5$ cm como máximo).

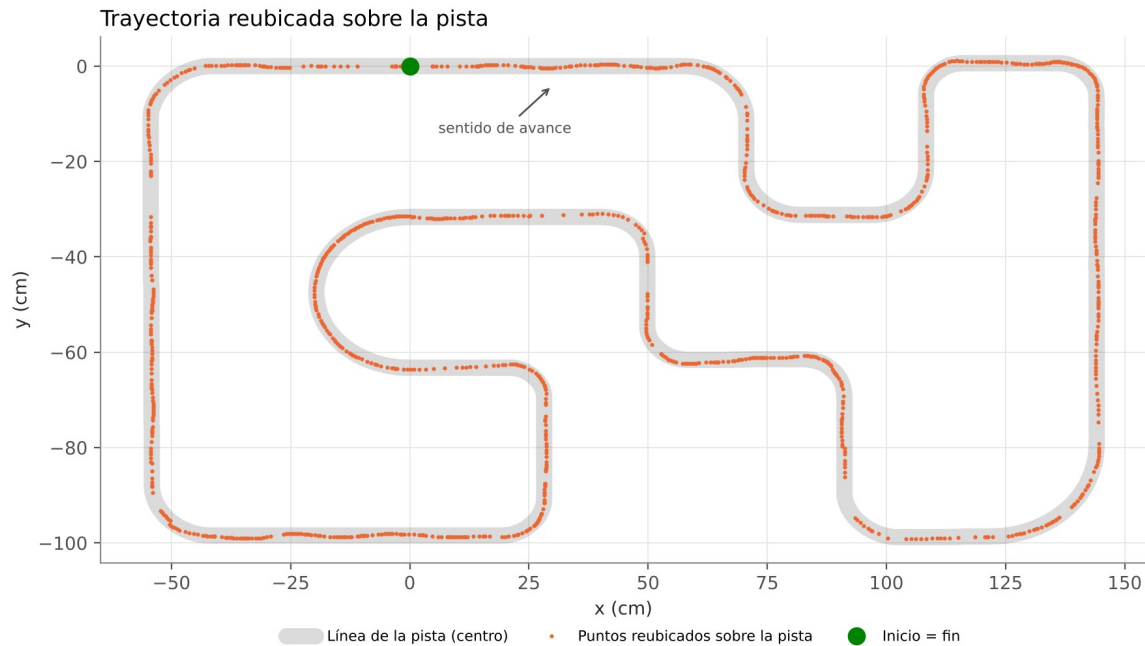


Figura 5. Trayectoria reubicada sobre la pista con emparejamiento con el mapa. Inicio y fin coinciden en la línea transversal.

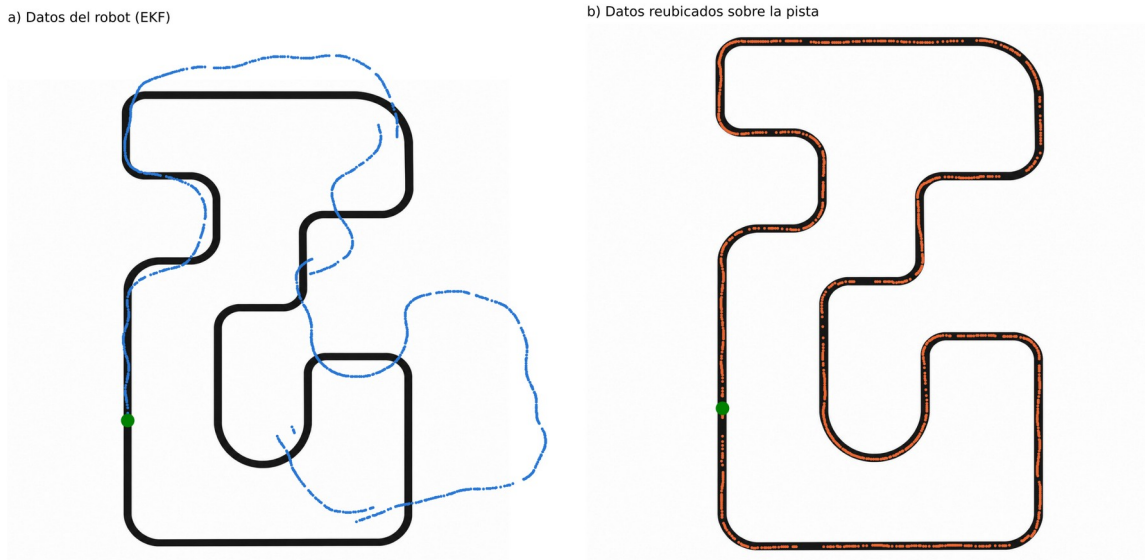


Figura 6. Datos dibujados sobre la imagen original del plano (con la escala de cada eje corregida): a) datos del EKF; b) datos reubicados sobre la línea.

El resultado (archivo `datos_reubicados_pista.csv`) contiene para cada punto su distancia recorrida, la posición estimada por el EKF, la posición reubicada en la pista y el error entre ambas. Es importante aclarar que este paso usa información externa (el plano de la pista) y se hace después del recorrido. Por eso **no reemplaza** el archivo `datos.txt`, que es la estimación real del robot; sirve para visualizar el recorrido sobre el plano y para medir el error del filtro.

7.4. ANÁLISIS DE RESULTADOS

Las causas principales de la diferencia entre la estimación y la pista son:

- **Errores de rumbo en las curvas:** un error de pocos grados en un tramo se convierte en un error de posición que crece con todo lo que se recorre después. La pista tiene muchas curvas de 90° seguidas y el seguidor las toma con oscilación, por lo que el giroscopio acumula errores de escala y las ruedas patinan.
- **Las mediciones del LiDAR también son relativas:** el ICP mide cuánto se movió el robot entre barridos, no dónde está en la pista. Al fusionarlo, el EKF reduce el crecimiento del error, pero sin una referencia absoluta (un mapa o una marca conocida) el error acumulado no se puede eliminar del todo.
- **Entorno poco estructurado para el LiDAR:** la pista está en el piso de un espacio abierto; el LiDAR ve paredes lejanas, mesas y personas que se mueven. Con pocas superficies útiles, algunos emparejamientos son pobres y aparecen correcciones bruscas de 4 a 8,5 cm (los saltos de la Figura 2).
- **Escala de la odometría:** la distancia registrada fue unos 13 % menor que el perímetro de la línea central. El radio de rueda efectivo, el número de cuentas por vuelta y el deslizamiento sobre la lona afectan directamente esta escala. Un error de pocos por ciento en el radio cambia el tamaño de toda la trayectoria.

Aun así, la forma general del recorrido sí se reconoce (salida, escalón superior, curva grande a la derecha, zona central y regreso por la parte inferior), el giro total de 360° es correcto y el sistema fue robusto: el robot siguió la línea, detectó la salida y la meta y guardó los datos sin perder muestras.

8. CONCLUSIONES

- Se implementó una solución completa: seguidor de línea con control de velocidad por rueda, detección de la línea transversal, predicción con encoders + giroscopio, scan matching PL-ICP con el LiDAR y un filtro de Kalman extendido con Q dependiente del movimiento, R entregada por el ICP y validación robusta de las mediciones.
- Separar la marcha (100 Hz) de la estimación y procesar el LiDAR en modo diferido fue clave para que el robot recorriera la pista de forma estable con los recursos limitados de MicroPython.
- La estimación coincide con la pista en los primeros 170 cm (error medio de 4,5 cm) y reproduce el giro total de la vuelta, pero acumula un error de cierre de 57,7 cm (7,7 % de la distancia), causado sobre todo por errores de rumbo en las curvas y por correcciones del LiDAR de baja calidad en un entorno con pocas superficies.
- Para comparar los datos con la pista se deben poner en el mismo sistema de coordenadas, con la escala real de la pista (99,4 cm × 198,5 cm; el plano en imagen no está a escala) y el mismo punto de salida. Con el emparejamiento con el mapa, los 1090 puntos quedan ubicados sobre el trayecto real.

8.1. MEJORAS PROPUESTAS

- Usar la línea transversal de meta como **cierre de lazo**: se sabe que el robot vuelve a la pose inicial, así que esa información puede corregir toda la trayectoria al final.
- Calibrar el factor de escala del giroscopio con 10 giros completos y el radio de rueda con una recta medida de 1 m, en la misma superficie de la pista.
- Colocar paredes o cajas alrededor de la pista para que el LiDAR tenga superficies estables, y guardar los barridos para ajustar R con más datos.
- Usar las esquinas de la pista (cambios de rumbo de 90°) como referencias del mapa para corregir la posición durante el recorrido.

9. REFERENCIAS

Censi, A. (2008). An ICP variant using a point-to-line metric. *IEEE International Conference on Robotics and Automation (ICRA)*, 19–25.

Thrun, S., Burgard, W. y Fox, D. (2005). *Probabilistic Robotics*. MIT Press.

Siegwart, R., Nourbakhsh, I. y Scaramuzza, D. (2011). *Introduction to Autonomous Mobile Robots* (2.^a ed.). MIT Press.

Universidad EAFIT (2026). *Reto: Mapeo de Trayectoria con Robot XRP* – Guía para el estudiante, Actuadores y Sensores 2026-2.

SparkFun / XRP. *XRPLib* – documentación de la librería del robot XRP.

10. ANEXO: ARCHIVOS ENTREGADOS

Tabla 6. Archivos del reto.

Archivo	Contenido
reto1_xrp.py	Programa del robot (MicroPython, XRPLib): marcha, LiDAR, ICP, EKF y registro.
datos.txt	Trayectoria (x, y) en cm estimada por el EKF del robot (formato de la guía).
datos_reubicados_pista.csv	Para cada punto: distancia recorrida, posición del EKF, posición reubicada sobre la pista y error.
datos_reubicados.txt	Trayectoria reubicada sobre la pista en formato x, y.
trayecto_sobre_pista.png	Recorrido dibujado sobre el plano de la pista.